

# 0.8.1. Tout l'alphabet, avec deux boucles — de base, ailleurs

Le 0.8 trois lignes plus bas : tout l'alphabet sur les lignes 5 et 6.

## 1. Ce qu'il faut comprendre

C'est le 0.8, avec un seul changement : les **lignes** : 5 et 6 au lieu de 0 et 1, dans les deux `poser`.

Le reste ne bouge pas : A à T sur une ligne, U à Z sur la suivante.

C'est la version de base : une seule chose, à la deuxième place. Le 0.8.2 met les deux ensemble.

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1400 octets** de cartouche.

### 1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // Le changement : les lignes 0 et 1 → 5 et 6.
    for (uint8_t i = 0; i < 20; i++) {    // A à T : ligne 5
        poser(i, 5, ALPHABET[i]);
    }
    for (uint8_t i = 20; i < 26; i++) {    // U à Z : ligne 6
        poser(i - 20, 6, ALPHABET[i]);
    }

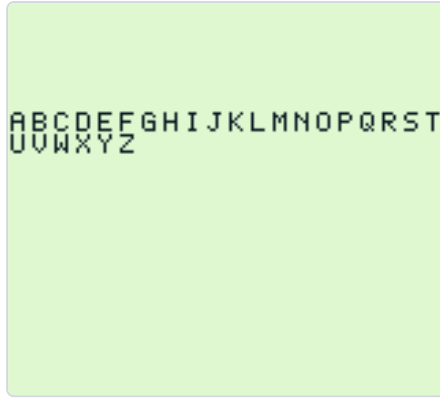
    while (true) {
        image();
    }
}
```

## 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

## 4. Ce qu'on doit voir

Tout l'alphabet sur les lignes 5 et 6.

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ ligne 5 : A à T

✓ ligne 6 : U à Z

Ensuite : **0.8.2. Tout l'alphabet, avec deux boucles — doublé, deux positions** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++